

PART A pseudocodes

ENQUEUE(student):

IF rear == SIZE-1 THEN

 DISPLAY "Queue is full"

ELSE IF front == -1 THEN

 front = 0

 rear = 0

 queue[rear] = student

ELSE

 rear = rear + 1

 queue[rear] = student

ENDIF

DEQUEUE():

IF front == -1 THEN

 DISPLAY "Queue is empty"

ELSE

 student = queue[front]

 IF front == rear THEN

 front = -1

 rear = -1

 ELSE

 front = front + 1

 ENDIF

```
    DISPLAY "Dequeued: ", student  
ENDIF
```

PEEK():

```
IF front == -1 THEN  
    DISPLAY "Queue is empty"  
ELSE  
    RETURN queue[front]  
ENDIF
```

DISPLAY():

```
IF front == -1 THEN  
    DISPLAY "Queue is empty"  
ELSE  
    FOR i = front TO rear  
        PRINT queue[i]  
    ENDFOR  
ENDIF
```

Part A2 pseudocodes

insertAtBeginning(student):

```
    newNode = createNode(student)  
    newNode.next = head  
    head = newNode
```

insertAtEnd(student):

```
newNode = createNode(student)
```

```
IF head == null THEN
```

```
    head = newNode
```

```
ELSE
```

```
    temp = head
```

```
    WHILE temp.next != null
```

```
        temp = temp.next
```

```
    ENDWHILE
```

```
    temp.next = newNode
```

```
ENDIF
```

```
insertAtPosition(student, pos):
```

```
    newNode = createNode(student)
```

```
    temp = head
```

```
    FOR i = 1 TO pos-1
```

```
        temp = temp.next
```

```
    ENDFOR
```

```
    newNode.next = temp.next
```

```
    temp.next = newNode
```

```
deleteStudent(studentNumber):
```

```
    temp = head
```

```
    IF temp.num == studentNumber THEN
```

```
        head = temp.next
```

```
    ELSE
```

```
        WHILE temp.next.num != studentNumber
```

```
        temp = temp.next
    ENDWHILE
    temp.next = temp.next.next
ENDIF
```

searchStudent(studentNumber):

```
temp = head
WHILE temp != null
    IF temp.num == studentNumber THEN
        RETURN temp
    ENDIF
    temp = temp.next
ENDWHILE
RETURN "Not found"
```

displayStudents():

```
temp = head
WHILE temp != null
    PRINT temp.num, temp.name, temp.service, temp.time
    temp = temp.next
ENDWHILE
```

Part A3 pseudocodes

push(x):

```
top = top + 1
stack[top] = x
```

pop():

 x = stack[top]

 top = top - 1

 RETURN x

peek():

 RETURN stack[top]

evaluatePostfix(expression):

 FOR each token in expression

 IF token is number THEN

 push(token)

 ELSE

 b = pop()

 a = pop()

 result = applyOperator(a, b, token)

 push(result)

 ENDIF

 ENDFOR

 RETURN pop()

Part A4

calculateStatistics(times[]):

 totalStudents = length(times)

 totalTime = 0

highest = times[0]

lowest = times[0]

countLong = 0

FOR each t in times

totalTime = totalTime + t

IF t > highest THEN highest = t

IF t < lowest THEN lowest = t

IF t > 10 THEN countLong = countLong + 1

ENDFOR

average = totalTime / totalStudents

PRINT "Total students:", totalStudents

PRINT "Total service time:", totalTime

PRINT "Average service time:", average

PRINT "Highest:", highest

PRINT "Lowest:", lowest

PRINT "Services >10min:", countLong

Part A5

Why a queue is appropriate A1

A queue enforces First-In-First-Out ordering directly through its two defining operations: enqueue always adds at the rear, and dequeue always removes from the front. This matches the real-world rule at the service centre — the student who has been waiting longest is served next — without the program needing any extra bookkeeping such as timestamps or manual sorting by arrival time.

Why a linked list is appropriate A2

unlike an array, a linked list does not need to shift every following element when a node is inserted or removed partway through the list. Only the pointers immediately around the affected node change, which keeps insertion and deletion efficient even as the number of records grows.

Why a stack is appropriate A3

Postfix evaluation naturally processes operands and partial results in Last-In-First-Out order — the two most recently pushed values are always the operands for the next operator encountered. A stack's push/pop operations give direct $O(1)$ access to exactly those two values without searching.

Why an array is appropriate A4

Once a student has been served, their service time is a fixed, ordered value that only needs to be scanned — not inserted or removed from the middle. Direct indexing lets every statistic (total, average, highest, lowest, count-over-10) be computed with a single manual pass over contiguous memory, which is simpler and faster than a linked structure would be for this read-only traversal pattern.